Formularios, validación, persistencia e integración en JavaScript

Enfoque de la sesión

En esta sesión vas a trabajar uno de los escenarios más habituales del desarrollo frontend: **recoger información introducida por la persona usuaria, validarla, decidir qué hacer con ella y almacenarla si es necesario**. El objetivo no es solo saber leer un formulario, sino comprender un flujo mucho más completo: entrada de datos, validación, persistencia y organización razonable de la lógica de interfaz.

1. Introducción

En la sesión anterior trabajaste con eventos, delegación e interacción. Eso te permitió construir una interfaz que no solo mostraba información, sino que también reaccionaba a acciones de la persona usuaria. Sin embargo, una aplicación web real no se limita a responder a clics o a cambiar estados visuales. También necesita **recibir datos**, **comprobar si son válidos**, **decidir si pueden procesarse** y, en muchos casos, **conservarlos**.

Los formularios son uno de los mecanismos clásicos de entrada de datos en la web. A través de ellos se registran usuarios, se crean tareas, se envían mensajes, se realizan búsquedas, se editan perfiles o se ajustan configuraciones de una aplicación.

Pero trabajar con formularios no consiste solo en capturar texto o leer el valor de un input. También implica:

- interceptar el envío;
- validar la información;
- decidir si el flujo puede continuar;
- mostrar mensajes adecuados;
- limpiar o transformar datos de entrada;
- y, si hace falta, guardar información en el navegador.

Por eso esta sesión reúne varias piezas que encajan muy bien entre sí:

- formularios;
- evento `submit`;
- lectura de valores;
- `FormData`;

- validación nativa;
- validación personalizada;
- `checkValidity()` y `reportValidity()`;
- `localStorage` y `sessionStorage`;
- JSON como formato de almacenamiento estructurado;
- y una organización mínima del flujo.

Esta sesión actúa como cierre del bloque de JavaScript porque integra muchas ideas trabajadas anteriormente:

- DOM;
- eventos;
- transformación de datos;
- asincronía;
- organización del código;
- y pensamiento estructurado sobre el flujo de una aplicación.

---

2. Formularios en HTML y su papel en JavaScript

2.1. Qué es un formulario

Un formulario es una estructura HTML pensada para recoger información introducida por la persona usuaria.

Ejemplo básico:

```html
<form id="formulario-contacto">
  <label for="nombre">Nombre</label>
  <input id="nombre" name="nombre" type="text">

  <label for="email">Correo electrónico</label>
  <input id="email" name="email" type="email">

  <button type="submit">Enviar</button>
</form>
```

2.2. Qué aporta JavaScript sobre un formulario

JavaScript permite:

- leer el valor de los campos;

- validar información;
- impedir un envío no deseado;
- mostrar errores o mensajes;
- construir respuestas visuales dinámicas;
- guardar datos localmente;
- procesar la información antes de enviarla o almacenarla.

2.3. El formulario como punto de integración

En un formulario se cruzan varias ideas importantes del bloque:

- DOM;
- eventos;
- validación;
- transformación de datos;
- persistencia;
- actualización visual.

> **Idea clave**
>
> Un formulario no es solo un conjunto de campos.
> Es un punto donde la interfaz, la lógica y los datos deben trabajar coordinadamente.

Referencias específicas del subtema

- MDN Web Docs — HTML forms guide
- MDN Web Docs — `<form>`
- MDN Web Docs — `<input>`

---

3. Escuchar el envío con `submit`

3.1. El evento `submit`

Cuando una persona usuaria envía un formulario, se dispara el evento `submit`.

```
const formulario = document.querySelector("#formulario-contacto");

formulario.addEventListener("submit", () => {
```

```
  console.log("Formulario enviado");
});
```

3.2. Comportamiento por defecto

Por defecto, el envío de un formulario puede provocar una recarga de la página o una navegación, dependiendo de su configuración.

En muchas miniaplicaciones construidas con JavaScript necesitas interceptar ese comportamiento para poder controlar tú el flujo.

3.3. `preventDefault()`

Para evitar el comportamiento por defecto, se utiliza `preventDefault()` dentro del listener.

```
const formulario = document.querySelector("#formulario-contacto");

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();
  console.log("Envío interceptado");
});
```

3.4. Por qué es importante

Esto te permite:

- validar antes de enviar;
- decidir si realmente debe procesarse;
- evitar recargas innecesarias;
- controlar tú la lógica del flujo.

> **Idea clave**
>
> `preventDefault()` no “rompe” el formulario.
> Simplemente te permite tomar el control del proceso con JavaScript.

Referencias específicas del subtema

- MDN Web Docs — HTMLFormElement: `submit` event
- MDN Web Docs — `Event.preventDefault()`

---

4. Leer valores de los campos

4.1. Acceder al valor de un input

Puedes seleccionar un campo y leer su valor mediante la propiedad `value`.

```
<input id="nombre" type="text">
```

```
const inputNombre = document.querySelector("#nombre");

console.log(inputNombre.value);
```

4.2. Lectura de varios campos

```
<form id="formulario-contacto">
  <input id="nombre" type="text">
  <input id="email" type="email">
  <button type="submit">Enviar</button>
</form>
```

```
const formulario = document.querySelector("#formulario-contacto");
const inputNombre = document.querySelector("#nombre");
const inputEmail = document.querySelector("#email");

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();

  const nombre = inputNombre.value;
  const email = inputEmail.value;

  console.log(nombre);
  console.log(email);
});
```

4.3. Limpiar espacios innecesarios

En muchos casos conviene limpiar el texto con `trim()`.

```
const nombre = inputNombre.value.trim();
```

Esto resulta útil para evitar entradas aparentemente rellenas que en realidad contienen solo espacios.

4.4. Agrupar datos en un objeto

Una práctica muy útil consiste en reunir los valores del formulario en una estructura clara.

```
const datosFormulario = {
  nombre: inputNombre.value.trim(),
  email: inputEmail.value.trim()
};

console.log(datosFormulario);
```

Referencias específicas del subtema

- MDN Web Docs — `HTMLInputElement.value`
- MDN Web Docs — `String.prototype.trim()`

---

5. Lectura alternativa con `FormData`

5.1. Qué es `FormData`

`FormData` es una herramienta nativa del navegador que permite recoger los datos de un formulario de una forma más cómoda, especialmente cuando el formulario tiene varios campos.

En lugar de seleccionar manualmente cada input y leer su propiedad `value`, puedes construir un objeto `FormData` a partir del formulario completo y extraer después los valores que necesites.

5.2. Cuándo resulta útil

`FormData` resulta especialmente útil cuando:

- el formulario tiene varios campos;
- quieres evitar repetir demasiados `querySelector()` y accesos a `.value`;
- te interesa trabajar con una estructura pensada para recoger datos del formulario como conjunto;
- quieres preparar mejor el terreno para flujos de procesamiento posteriores.

5.3. Requisito importante: el atributo `name`

Para que `FormData` funcione correctamente, los campos del formulario deben tener definido el atributo `name`.

```
<form id="formulario-contacto">
  <input id="nombre" name="nombre" type="text">
  <input id="email" name="email" type="email">
  <button type="submit">Enviar</button>
</form>
```

Aquí, aunque los campos también tienen `id`, `FormData` utilizará los valores de `name` para identificar cada entrada.

> **Idea clave**
>
> `id` sirve muy bien para seleccionar elementos en el DOM.
> `name` es especialmente importante cuando trabajas con formularios y con `FormData`.

5.4. Ejemplo básico de uso

```
const formulario = document.querySelector("#formulario-contacto");

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();

  const datosFormulario = new FormData(formulario);

  const nombre = datosFormulario.get("nombre");
  const email = datosFormulario.get("email");

  console.log(nombre);
  console.log(email);
});
```

5.5. Relación con la lectura manual mediante `.value`

Esto no sustituye completamente a la lectura manual con `.value`, sino que ofrece una alternativa práctica.

Ambos enfoques son útiles:

Enfoque	Cuándo resulta más útil
`.value`	cuando quieres entender el acceso directo a un campo concreto o trabajar con pocos inputs
`FormData`	cuando quieres recoger de forma más cómoda los datos del formulario como conjunto

5.6. Precaución práctica

Aunque `FormData` facilita la recogida de valores, eso no elimina la necesidad de:

- limpiar entradas cuando haga falta;
- validar los datos;
- comprobar si los valores tienen el formato esperado.

Es decir, `FormData` ayuda a **recoger** la información, pero no sustituye la lógica de validación.

Referencias específicas del subtema

- MDN Web Docs — `FormData`
- MDN Web Docs — Using FormData objects
- MDN Web Docs — HTML forms guide

---

6. Validación nativa del navegador

6.1. Qué es la validación nativa

HTML ofrece mecanismos básicos de validación sin necesidad de escribir toda la lógica desde cero. Algunas restricciones pueden definirse directamente en los atributos del campo.

Ejemplo:

```
<form id="registro">
  <label for="email">Correo electrónico</label>
  <input id="email" type="email" required>

  <label for="edad">Edad</label>
  <input id="edad" type="number" min="18" required>
```

```
  <button type="submit">Registrarse</button>
</form>
```

Aquí el navegador ya sabe que:

- el correo es obligatorio;
- debe respetar el formato de un email;
- la edad debe estar presente;
- la edad mínima es 18.

6.2. Ventajas de la validación nativa

- reduce parte del trabajo repetitivo;
- se integra con el comportamiento del navegador;
- aporta una primera capa de control;
- resulta útil como base incluso cuando añades validación personalizada.

6.3. Limitaciones

La validación nativa no cubre todos los casos de negocio ni todas las reglas específicas de una aplicación. Por eso muchas veces se combina con validación personalizada.

> **Criterio recomendado**
>
> La validación nativa no debe verse como un recurso menor.
> Muy a menudo conviene aprovecharla como primera capa y complementar después con lógica propia.

Referencias específicas del subtema

- MDN Web Docs — Constraint validation
- MDN Web Docs — `<input>`
- MDN Web Docs — HTML attribute: `required`

---

7. `checkValidity()` y `reportValidity()`

7.1. `checkValidity()`

`checkValidity()` permite comprobar si un formulario o campo cumple las restricciones definidas.

```
const formulario = document.querySelector("#registro");

console.log(formulario.checkValidity());
```

Devuelve `true` si todo es válido y `false` si hay algún incumplimiento.

7.2. `reportValidity()`

`reportValidity()` además de comprobar la validez, solicita al navegador que muestre el mensaje visual correspondiente.

```
const formulario = document.querySelector("#registro");

formulario.reportValidity();
```

7.3. Uso en contexto de envío

```
const formulario = document.querySelector("#registro");

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();

  if (!formulario.checkValidity()) {
    formulario.reportValidity();
    return;
  }

  console.log("Formulario válido");
});
```

7.4. Qué aporta esta combinación

Esta estrategia permite:

- interceptar el flujo con JavaScript;
- aprovechar la validación nativa;
- evitar continuar con datos inválidos;
- organizar mejor el procesamiento posterior.

Referencias específicas del subtema

- MDN Web Docs — `HTMLFormElement.checkValidity()`

- MDN Web Docs — `HTMLFormElement.reportValidity()`

---

8. Validación personalizada

8.1. Por qué hace falta

No todas las reglas que necesitas pueden expresarse únicamente con atributos HTML. Por ejemplo:

- comprobar longitud mínima con una lógica concreta;
- impedir ciertos patrones;
- combinar varios campos;
- personalizar mensajes;
- verificar reglas de negocio.

8.2. Ejemplo sencillo

```html
<form id="registro-usuario">
  <input id="nombre-usuario" type="text" required>
  <button type="submit">Guardar</button>
</form>

<p id="error-usuario"></p>
```

```javascript
const formulario = document.querySelector("#registro-usuario");
const inputUsuario = document.querySelector("#nombre-usuario");
const errorUsuario = document.querySelector("#error-usuario");

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();

  const nombre = inputUsuario.value.trim();

  if (nombre.length < 3) {
    errorUsuario.textContent = "El nombre debe tener al menos 3 caracteres.";
    return;
  }

  errorUsuario.textContent = "";
  console.log("Formulario válido");
});
```

8.3. Validación nativa + personalizada

Muchas veces la mejor estrategia no es elegir una u otra, sino combinarlas:

1. comprobar primero si el formulario cumple las restricciones nativas;
2. aplicar después reglas específicas de negocio.

8.4. Importancia de los mensajes claros

Una validación útil no solo detecta un problema. También ayuda a entenderlo. Por eso conviene que los mensajes:

- indiquen qué campo falla;
- expliquen qué se espera;
- no sean ambiguos.

> **Idea clave**
>
> Validar no consiste solo en bloquear el envío.
> Consiste en guiar a la persona usuaria para que pueda corregir la entrada.

Referencias específicas del subtema

- MDN Web Docs — Constraint validation
- MDN Web Docs — `Node.textContent`

---

9. Validación de entradas y limpieza de datos

9.1. La validación no empieza ni termina en el navegador

Aunque en esta sesión trabajes en frontend, debes acostumbrarte a pensar que los datos de entrada deben tratarse con cuidado. Conviene:

- limpiar espacios;
- comprobar tipos esperados;
- validar formato;
- evitar asumir que todo lo introducido es correcto.

9.2. Ejemplo con conversión numérica

```
<input id="edad" type="number">
```

```
const inputEdad = document.querySelector("#edad");
const edad = Number(inputEdad.value);

console.log(edad);
```

9.3. Comprobación simple

```
if (Number.isNaN(edad)) {
  console.log("La edad no es válida");
}
```

9.4. Importancia de distinguir ausencia, error y valor válido

No es lo mismo:

- que un campo esté vacío;
- que el valor tenga formato incorrecto;
- que exista un valor válido.

Cuanto más claro tengas eso, mejor diseñarás tus formularios.

Referencias específicas del subtema

- MDN Web Docs — `Number()`
- MDN Web Docs — `Number.isNaN()`

---

10. Persistencia local con `localStorage`

10.1. Qué es `localStorage`

`localStorage` es un mecanismo del navegador que permite guardar pares clave-valor de forma persistente. Es decir, la información puede seguir disponible aunque se recargue la página o se cierre y vuelva a abrir el navegador.

10.2. Guardar un valor simple

```
localStorage.setItem("tema", "oscuro");
```

10.3. Recuperar un valor

```
const tema = localStorage.getItem("tema");
console.log(tema);
```

10.4. Eliminar un valor

```
localStorage.removeItem("tema");
```

10.5. Vaciar todo el almacenamiento local

```
localStorage.clear();
```

10.6. Cuándo resulta útil

`localStorage` es útil para:

- conservar configuraciones;
- guardar tareas o datos simples;
- mantener preferencias de interfaz;
- almacenar resultados locales de una miniaplicación.

> ⚠ **Precaución importante**
>
> `localStorage` no es una base de datos completa ni un lugar adecuado para cualquier tipo de información sensible.
> En esta fase debes entenderlo como una herramienta práctica para persistencia local sencilla.

Referencias específicas del subtema

- MDN Web Docs — `Window.localStorage`

---

11. Persistencia temporal con `sessionStorage`

11.1. Qué es `sessionStorage`

`sessionStorage` funciona de manera parecida a `localStorage`, pero su duración está ligada a la sesión de la pestaña o ventana actual.

11.2. Ejemplo básico

```
sessionStorage.setItem("filtroActual", "completadas");

const filtro = sessionStorage.getItem("filtroActual");
console.log(filtro);
```

11.3. Diferencia general respecto a `localStorage`

Mecanismo	Duración habitual
`localStorage`	persiste entre sesiones
`sessionStorage`	persiste durante la sesión actual de la pestaña

11.4. Cuándo puede ser útil

`sessionStorage` puede encajar cuando quieres conservar datos temporales mientras la persona usuaria sigue en una sesión concreta, pero no necesitas que permanezcan después.

Referencias específicas del subtema

- MDN Web Docs — `Window.sessionStorage`

---

12. JSON y almacenamiento estructurado

12.1. El problema

Tanto `localStorage` como `sessionStorage` almacenan valores en formato textual. Si quieres guardar arrays u objetos, necesitas convertirlos.

12.2. Guardar un objeto

```
const usuario = {
  nombre: "Laura",
  email: "laura@email.com"
};
```

```
localStorage.setItem("usuario", JSON.stringify(usuario));
```

12.3. Recuperar un objeto

```
const usuarioGuardado = localStorage.getItem("usuario");
const usuario = JSON.parse(usuarioGuardado);

console.log(usuario.nombre);
```

12.4. Ejemplo con array

```
const tareas = [
  { titulo: "Estudiar", completada: false },
  { titulo: "Practicar", completada: true }
];

localStorage.setItem("tareas", JSON.stringify(tareas));

const tareasRecuperadas = JSON.parse(localStorage.getItem("tareas"));
console.log(tareasRecuperadas);
```

12.5. Precaución al recuperar datos

Si no existe la clave, `getItem()` devuelve `null`. Conviene tenerlo en cuenta.

```
const tareasGuardadas = localStorage.getItem("tareas");
const tareas = tareasGuardadas ? JSON.parse(tareasGuardadas) : [];
```

> **Idea clave**
>
> Cuando guardas arrays u objetos en almacenamiento web, en realidad los conviertes en texto JSON y luego los reconstruyes al recuperarlos.

Referencias específicas del subtema

- MDN Web Docs — `JSON.stringify()`
- MDN Web Docs — `JSON.parse()`
- MDN Web Docs — `Window.localStorage`

---

13. Integrar formulario, validación y persistencia

13.1. Por qué conviene ver estas piezas juntas

En una aplicación real, formularios, validación y persistencia suelen aparecer conectados:

- la persona usuaria introduce información;
- el formulario se valida;
- se decide si el flujo puede continuar;
- y, si todo es correcto, se guarda o se utiliza la información.

13.2. Qué cambia respecto a ver cada pieza por separado

Si estudias cada bloque de forma aislada, puedes comprender la sintaxis. Pero cuando los integras, entiendes mejor el flujo real de una miniaplicación.

13.3. Flujo general razonable

Un flujo habitual sería este:

1. escuchar `submit`;
2. interceptar el comportamiento por defecto;
3. leer datos;
4. limpiar entradas;
5. validar;
6. construir una estructura clara;
7. guardar si procede;
8. actualizar la interfaz.

> **Idea clave**
>
> La calidad de una miniaplicación no depende solo de que “el formulario funcione”, sino de cómo se organiza todo el proceso alrededor de ese formulario.

Referencias específicas del subtema

- MDN Web Docs — HTML forms guide
- MDN Web Docs — Constraint validation
- MDN Web Docs — `Window.localStorage`

---

14. Organización mínima del código en este tipo de flujos

14.1. El problema de mezclarlo todo

Cuando una misma función:

- escucha el envío;
- valida;
- muestra mensajes;
- construye objetos;
- guarda datos;
- limpia campos;
- y modifica el DOM;

el código puede volverse difícil de mantener.

14.2. Una separación razonable

Aunque la aplicación sea pequeña, conviene empezar a distinguir:

- lectura o recogida de datos;
- validación;
- persistencia;
- actualización visual.

14.3. Ejemplo conceptual

```
function obtenerDatosFormulario(formulario) {
  const datos = new FormData(formulario);

  return {
    nombre: datos.get("nombre")?.trim(),
    email: datos.get("email")?.trim()
  };
}

function validarUsuario(usuario) {
  return usuario.nombre.length >= 3 && usuario.email.includes("@");
}

function guardarUsuario(usuario) {
  const usuariosGuardados = localStorage.getItem("usuarios");
  const usuarios = usuariosGuardados ? JSON.parse(usuariosGuardados) : [];
```

```
  usuarios.push(usuario);
  localStorage.setItem("usuarios", JSON.stringify(usuarios));
}
```

14.4. Qué aporta esta separación

- la lógica es más fácil de leer;
- la validación puede entenderse mejor;
- la persistencia queda más localizada;
- el flujo global resulta más mantenible.

Referencias específicas del subtema

- MDN Web Docs — Functions
- MDN Web Docs — `FormData`
- MDN Web Docs — `Window.localStorage`

---

15. Análisis técnico avanzado

15.1. Validar bien mejora la calidad de toda la aplicación

La validación no debe verse como una barrera molesta al final del proceso. Es una parte central de la calidad de una interfaz y de la fiabilidad del flujo de datos.

15.2. Validación nativa y personalizada no se excluyen

Una aplicación sólida puede aprovechar las restricciones del navegador y, al mismo tiempo, añadir reglas propias del dominio. Ambas capas se complementan bien cuando se usan con criterio.

15.3. Almacenar no es lo mismo que modelar

Guardar información en `localStorage` o `sessionStorage` no resuelve por sí solo un buen diseño de datos. La estructura que guardas sigue importando, así como la forma en que la reconstruyes después.

15.4. JSON actúa como puente textual

JSON no es el dato “vivo” de tu aplicación, sino una representación textual que te permite almacenar o intercambiar información estructurada.

15.5. `FormData` no sustituye la comprensión del formulario

`FormData` es muy útil, pero no debe ocultar cómo funciona realmente un formulario, cómo se leen sus campos ni qué papel cumplen atributos como `name`, `required`, `type` o `min`.

15.6. La integración de estas piezas prepara muy bien el paso a React

Aunque el enfoque declarativo de React cambie la arquitectura, muchos de los problemas seguirán siendo los mismos:

- leer entradas;
- validar;
- almacenar o enviar información;
- actualizar la interfaz;
- y organizar el flujo con claridad.

Referencias específicas del subtema

- MDN Web Docs — Constraint validation
- MDN Web Docs — `Window.localStorage`
- MDN Web Docs — JSON
- MDN Web Docs — `FormData`

---

16. Ejemplo integrador de sesión

A continuación tienes un ejemplo que reúne varios conceptos trabajados.

```html
<form id="formulario-registro">
  <label for="nombre">Nombre</label>
  <input id="nombre" name="nombre" type="text" required>

  <label for="email">Email</label>
  <input id="email" name="email" type="email" required>

  <button type="submit">Guardar</button>
</form>

<p id="mensaje"></p>
```

```javascript
const formulario = document.querySelector("#formulario-registro");
const mensaje = document.querySelector("#mensaje");
```

```
function obtenerDatosFormulario(formulario) {
  const datos = new FormData(formulario);

  return {
    id: Date.now(),
    nombre: datos.get("nombre")?.trim(),
    email: datos.get("email")?.trim()
  };
}

function validarUsuario(usuario) {
  return usuario.nombre.length >= 3;
}

function guardarUsuario(usuario) {
  const usuariosGuardados = localStorage.getItem("usuarios");
  const usuarios = usuariosGuardados ? JSON.parse(usuariosGuardados) : [];

  usuarios.push(usuario);
  localStorage.setItem("usuarios", JSON.stringify(usuarios));
}

formulario.addEventListener("submit", (evento) => {
  evento.preventDefault();

  if (!formulario.checkValidity()) {
    formulario.reportValidity();
    return;
  }

  const usuario = obtenerDatosFormulario(formulario);

  if (!validarUsuario(usuario)) {
    mensaje.textContent = "El nombre debe tener al menos 3 caracteres.";
    return;
  }

  guardarUsuario(usuario);

  mensaje.textContent = "Usuario guardado correctamente.";
  formulario.reset();
});
```

Qué se está aplicando aquí

Elemento	Uso en el ejemplo
`submit`	interceptar el envío del formulario
`preventDefault()`	evitar el comportamiento por defecto
`FormData`	recogida de datos del formulario
`checkValidity()` y `reportValidity()`	validación nativa
validación personalizada	longitud mínima del nombre
`trim()`	limpieza de entradas
`localStorage`	persistencia local
JSON	serialización y recuperación de datos
`reset()`	limpieza final del formulario

---

17. Errores frecuentes al comenzar

> ⚠ **Errores muy habituales**
>
> Estos fallos aparecen con mucha frecuencia al empezar a trabajar con formularios, validación y persistencia.

17.1. Olvidar `preventDefault()` cuando querías controlar el flujo con JavaScript

Si no interceptas el envío, la página puede recargarse y perderás el control del proceso que querías ejecutar.

17.2. Leer valores sin limpiar ni validar

Un campo puede parecer relleno y, sin embargo, contener solo espacios. Por eso `trim()` resulta tan útil en muchos escenarios.

17.3. Confiar solo en la validación nativa o solo en la personalizada

Ambas tienen valor, pero cumplen papeles distintos. Lo más razonable suele ser combinarlas según el caso.

17.4. Usar `FormData` sin haber definido correctamente los atributos `name`

Si el campo no tiene `name`, `FormData` no podrá recuperarlo como esperas.

17.5. Guardar objetos en `localStorage` sin convertirlos a JSON

```
const usuario = { nombre: "Ana" };
localStorage.setItem("usuario", usuario);
```

Eso no almacena el objeto como esperas. Necesitas `JSON.stringify()`.

17.6. Hacer `JSON.parse()` sobre un valor inexistente sin comprobarlo

Si la clave no existe, `getItem()` devuelve `null`. Conviene contemplarlo antes de parsear.

17.7. Mezclar validación, almacenamiento, mensajes y lectura de datos sin estructura mínima

Aunque al principio todo quepa en un único listener, conviene empezar a detectar cuándo una solución necesita separar responsabilidades.

---

18. Síntesis de la sesión

En esta sesión has trabajado uno de los bloques más aplicados del desarrollo frontend:

- formularios como mecanismo de entrada de datos;
- evento `submit` y control del flujo con `preventDefault()`;
- lectura de valores con `.value`;
- recogida alternativa de datos con `FormData`;
- validación nativa mediante atributos HTML;
- uso de `checkValidity()` y `reportValidity()`;
- validación personalizada para reglas específicas;
- limpieza y validación de entradas;
- persistencia con `localStorage` y `sessionStorage`;
- conversión entre estructuras JavaScript y texto JSON;
- integración de formularios, validación y almacenamiento;
- y una organización mínima del flujo.

Esta sesión es especialmente importante porque conecta entrada de datos, validación y persistencia. Esa combinación te prepara para trabajar con más criterio en cualquier interfaz que necesite recoger información de la persona usuaria y procesarla con claridad.

✓ **Al terminar esta sesión deberías ser capaz de...**

- interceptar el envío de un formulario con `submit`;
- usar `preventDefault()` para controlar el flujo;
- leer y limpiar valores de entrada;
- aprovechar validación nativa con `checkValidity()` y `reportValidity()`;
- añadir validación personalizada sencilla;
- utilizar `FormData` para recoger datos del formulario como conjunto;
- guardar y recuperar información con `localStorage` y `sessionStorage`;
- convertir datos a JSON y reconstruirlos después;
- comprender cómo se integra el flujo completo de lectura, validación, persistencia y actualización visual.

---

19. Referencias documentales generales

Referencias oficiales y de consulta

- MDN Web Docs — HTML forms guide
- MDN Web Docs — `<form>`
- MDN Web Docs — `<input>`
- MDN Web Docs — HTMLFormElement: `submit` event
- MDN Web Docs — `Event.preventDefault()`
- MDN Web Docs — Constraint validation
- MDN Web Docs — `HTMLFormElement.checkValidity()`
- MDN Web Docs — `HTMLFormElement.reportValidity()`
- MDN Web Docs — `HTMLInputElement.value`
- MDN Web Docs — `FormData`
- MDN Web Docs — Using FormData objects
- MDN Web Docs — `Window.localStorage`
- MDN Web Docs — `Window.sessionStorage`
- MDN Web Docs — JSON
- MDN Web Docs — `JSON.stringify()`

- MDN Web Docs — `JSON.parse()`
- MDN Web Docs — `String.prototype.trim()`
- MDN Web Docs — `Number()`
- MDN Web Docs — `Number.isNaN()`

Recomendación de estudio

Al repasar esta sesión, no te limites a comprobar que el formulario “funciona”. Intenta identificar cada fase del proceso: qué datos se leen, cómo se validan, cuándo se almacenan, en qué formato se guardan y qué parte del flujo decide si puede continuar o no. Entender bien esa secuencia te preparará muy bien para cualquier interfaz donde la persona usuaria introduzca información y tu aplicación tenga que procesarla con criterio.